HIGH-YIELD REVIEW

JavaScript Exam Cheat Sheet

Built around the COITB JavaScript Professional Developer outline. The published outline gives the most weight to syntax, arrays, objects, functions, controls, and loops; the badge page also expects DOM, events, forms, APIs, JSON, async code, and modules.

Exam format	90 items / 90 minutes / multiple-choice and scenarios
Passing score	COITB badge page lists 73% or higher; outline page lists 75% - confirm live requirement
Top domains	Syntax 22.94% | Arrays 10.27% | Objects 9.61%
Pacing	About 60 seconds per question; trace values on scratch paper

How to use this: Read the rule, predict the result before revealing the explanation, then type the example from memory. Focus first on the highest-weight domains.

1. Syntax, Variables, and Types

Declarations

Keyword	Scope	Reassign?	Redeclare same scope?
const	block	No	No
let	block	Yes	No
var	function/global	Yes	Yes

```
const user = { name: 'Ada' };
user.name = 'Grace'; // allowed: object mutated
// user = {};         // TypeError: binding cannot change

let score = 0;
score += 10;
```

Best default: Use const unless the variable binding must be reassigned. Use let when reassignment is intentional. Avoid var in modern code unless you specifically need its function-scoped behavior.

Primitive and reference values

- Seven primitive types: `string`, `number`, `bigint`, `boolean`, `undefined`, `symbol`, and `null`.
- Objects include ordinary objects, arrays, functions, dates, maps, sets, and more.
- Primitives compare by value. Objects compare by identity/reference.
- `typeof null` is `'object'` (a historic quirk). `Array.isArray(x)` reliably detects arrays.

```
typeof 42             // 'number'
typeof 'hi'           // 'string'
typeof undefined      // 'undefined'
typeof null           // 'object' (quirk)
Array.isArray([])      // true
Number.isNaN(NaN)     // true
```

null, undefined, and NaN

- `undefined`: missing/unassigned value. `null`: intentional absence by convention.
- `NaN` means Not-a-Number but its type is number. It is the only value not equal to itself; prefer `Number.isNaN`.

2. Operators, Equality, and Coercion

Category	Operators / rule
Arithmetic	+ - * / % **; exponentiation is **
Assignment	=, +=, -=, *=, /=, ??=
Comparison	< <= > >=; may coerce when types differ
Strict equality	=== and !== compare type and value; preferred
Loose equality	== and != perform coercion; know traps
Logical	&&, || return operands; ! returns boolean
Nullish	a ?? b uses b only when a is null or undefined
Optional chain	obj?.prop or fn?.() stops on null/undefined
Ternary	condition ? valueIfTrue : valueIfFalse

```
0 || 10       // 10 (0 is falsy)
0 ?? 10       // 0  (0 is not nullish)
'' || 'x'     // 'x'
'' ?? 'x'     // ''
'5' + 2       // '52' (string concatenation)
'5' - 2       // 3 (numeric coercion)
5 === '5'     // false
5 == '5'      // true
```

Falsy values

Exactly these common values are falsy: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, and `NaN`. Empty arrays and empty objects are truthy.

Exam trap: `[] == false` is true because loose equality coerces values, but `[] === false` is false. Prefer strict equality.

Precedence sanity rule

Use parentheses when an expression mixes categories. Remember that function calls/member access happen before arithmetic; multiplication/division before addition/subtraction; comparisons before equality; && before ||; assignment is low precedence.

3. Control Flow and Loops

```
if (score >= 90) {
  grade = 'A';
} else if (score >= 80) {
  grade = 'B';
} else {
  grade = 'C or below';
}

switch (role) {
  case 'admin': access = 'all'; break;
  case 'editor': access = 'write'; break;
  default: access = 'read';
}
```

- `switch` cases compare using strict equality. Without `break` or `return`, execution falls through.
- Use `break` to exit the nearest loop or switch; use `continue` to skip to the next loop iteration.

Loop	Best fit
for	known counter/index or custom update
while	repeat while condition is true; may run zero times
do...while	body must run at least once
for...of	iterable values: arrays, strings, maps, sets
for...in	enumerable property keys; usually avoid for arrays

```
for (let i = 0; i < items.length; i++) {
  if (items[i] == null) continue;
  console.log(i, items[i]);
}

for (const value of items) console.log(value);
for (const key in user) console.log(key, user[key]);
```

Loop trap: An off-by-one bug often comes from using `i <= array.length`. The last valid index is `array.length - 1`.

4. Functions, Parameters, and Scope

```
function add(a, b = 0) {
  return a + b;
}

const multiply = (a, b) => a * b;
const makeUser = name => ({ name, active: true });

function sum(...nums) {
  return nums.reduce((total, n) => total + n, 0);
}
```

Concept	Rule
declaration	function declarations are hoisted with their body
expression	function value assigned to a variable; variable rules apply
arrow	concise; lexical this; no own arguments; not constructible
default parameter	used when argument is missing or undefined, not null
rest ...args	collects remaining arguments into a real array
callback	function passed to another function for later/immediate use
return	ends function; no explicit return means undefined

Scope and closures

- JavaScript uses lexical scope: where a function is defined determines which outer variables it can access.
- `let` and `const` are block-scoped. `var` is function-scoped.
- A closure is a function plus access to variables from its creation scope, even after the outer function returns.

```
function makeCounter() {
  let count = 0;
  return () => ++count;
}
const next = makeCounter();
next(); // 1
next(); // 2
```

Hoisting and TDZ

Declarations are processed before execution, but access rules differ. Function declarations can be called before their source line. `var` exists as `undefined` before assignment. `let` and `const` are in the temporal dead zone until their declaration executes.

Arrow trap: An arrow function does not bind its own `this`. That is useful inside callbacks, but an arrow is usually wrong for an object method that needs dynamic receiver-based this.

5. Arrays and Iterators

```
const nums = [3, 1, 4];
nums[0]              // 3
nums.at(-1)          // 4
nums.length          // 3
const copy = [...nums];
const combined = [...nums, 9];
const nested = [[1, 2], [3, 4]];
```

Mutating vs non-mutating

Mutates original	Returns / effect	Non-mutating alternative
push / pop	new length / removed last	[...a, x] / a.slice(0, -1)
unshift / shift	new length / removed first	[x, ...a] / a.slice(1)
splice	removed items; add/remove in place	toSpliced or slices
sort	same sorted array	toSorted or [...a].sort(...)
reverse	same reversed array	toReversed or [...a].reverse()

Iterator methods

Method	Returns	Use
forEach	undefined	side effect for every item
map	new array	transform every item
filter	new array	keep items whose callback is truthy
find	first value or undefined	locate one matching value
findIndex	index or -1	locate one matching index
some	boolean	does at least one match?
every	boolean	do all match?
reduce	single accumulated value	combine items
includes	boolean	contains value?

```
const users = [{name:'A', active:true}, {name:'B', active:fa
lse}];
const names = users.map(u => u.name);        // ['A', 'B']
const active = users.filter(u => u.active);  // [{...A}]
const found = users.find(u => u.name === 'B');
const count = users.reduce((n, u) => n + (u.active ? 1 : 0),
 0);
```

Sort trap: Default sort compares strings: `[2, 10, 1].sort()` becomes `[1, 10, 2]`. Numeric ascending requires `(a, b) => a - b`.

6. Objects, Destructuring, and Spread

```
const user = {
  id: 7,
  name: 'Ada',
  address: { city: 'London' },
  greet() { return `Hi ${this.name}`; }
};

user.name             // dot: known valid identifier
user['name']           // bracket: computed/dynamic key
const key = 'name'; user[key];
```

Object tools

Expression	Result / use
Object.keys(o)	array of own enumerable string keys
Object.values(o)	array of values
Object.entries(o)	array of [key, value] pairs
'x' in o	checks own or inherited property
Object.hasOwn(o, 'x')	checks own property
delete o.x	removes property
o?.address?.city	safe access; undefined if chain is nullish

```
const { name, address: { city }, role = 'user' } = user;
const [first, , third] = [10, 20, 30];
const renamed = { ...user, name: 'Grace' };
const merged = { ...defaults, ...options }; // later keys wi
n
```

Shallow copy: Object/array spread copies only one level. Nested objects still share references unless copied separately or cloned with an appropriate deep-clone strategy.

Classes - object syntax sugar

```
class Account {
  constructor(owner) { this.owner = owner; }
  describe() { return `Owner: ${this.owner}`; }
}
const a = new Account('Ada');
a instanceof Account; // true
```

7. DOM Selection and Manipulation

API	Purpose
document.querySelector(sel)	first matching Element or null
document.querySelectorAll(sel)	static NodeList of all matches
document.getElementById(id)	element by ID or null
document.createElement(tag)	create detached element
parent.append / prepend	insert nodes or strings
node.remove()	remove node
el.closest(sel)	nearest matching self/ancestor
el.matches(sel)	does element match selector?

```
const list = document.querySelector('#tasks');
const item = document.createElement('li');
item.textContent = 'Review arrays';
item.classList.add('task');
item.dataset.id = '42';
item.setAttribute('aria-current', 'true');
list.append(item);
```

Content, attributes, and style

- `textContent` treats content as text. `innerHTML` parses HTML and can create injection vulnerabilities with untrusted input.
- Use `classList.add/remove/toggle/contains` for classes; prefer classes over many inline style changes.
- `dataset.userId` maps to `data-user-id`. Dataset values are strings.
- Many DOM selectors can return null. Guard or use optional chaining before accessing properties.

Exam trap: `querySelectorAll` returns a static NodeList, not an Array. It supports forEach, but use `Array.from(list)` or spread when Array methods are required.

8. Events and Forms

```
const form = document.querySelector('form');
form.addEventListener('submit', event => {
  event.preventDefault();
  if (!form.checkValidity()) {
    form.reportValidity();
    return;
  }
  const data = Object.fromEntries(new FormData(form));
  console.log(data);
});
```

Event object

Member	Meaning
event.target	deepest element that initiated the event
event.currentTarget	element whose listener is currently running
preventDefault()	cancel default action when event is cancelable
stopPropagation()	stop propagation to later ancestors
key	keyboard key value such as Enter or Escape

Propagation and delegation

Events travel through capture -> target -> bubble phases. Most listeners run in the bubbling phase. Delegation places one listener on an ancestor and uses `event.target.closest(...)` to handle current and future descendants.

```
document.querySelector('.todo-list').addEventListener('click', e => {
  const button = e.target.closest('[data-remove]');
  if (!button) return;
  button.closest('li')?.remove();
});
```

- `input` fires as a value changes; `change` usually fires when a change is committed.
- Form controls need a `name` to appear in FormData or normal form submission.
- Client-side validation improves UX but never replaces server-side validation.

9. JSON, Promises, Async/Await, and Fetch

JSON

```
const text = JSON.stringify({ id: 7, active: true });
const value = JSON.parse(text);
// JSON supports objects, arrays, strings, numbers, booleans, and null.
// It does not preserve functions, undefined, Symbol, Map, Set, or Date types.
```

Promises and async

```
function wait(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

async function run() {
  try {
    await wait(100);
    return 'done'; // async function resolves to this value
  } catch (error) {
    console.error(error);
    throw error;
  } finally {
    console.log('cleanup');
  }
}
```

- A Promise is pending, then fulfilled or rejected. `then` handles fulfillment; `catch` handles rejection; `finally` runs either way.
- An `async` function always returns a Promise. `await` pauses only that async function, not the entire program.
- Use `Promise.all` for independent tasks that should run concurrently; it rejects when any input rejects.

Fetch pattern

```
async function getUsers() {
  const response = await fetch('/api/users');
  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  return response.json();
}

getUsers().then(renderUsers).catch(showError);
```

Fetch trap: fetch normally rejects for network failures, not HTTP 404/500 responses. Check `response.ok` or `response.status` yourself. Also remember that `response.json()` returns a Promise.

10. Modules, Storage, and Errors

ES modules

```
// math.js
export const pi = 3.14159;
export default function add(a, b) { return a + b; }

// app.js
import add, { pi as circlePi } from './math.js';
console.log(add(circlePi, 1));
```

- Use `<script type='module' src='app.js'></script>`. Modules are deferred automatically and have their own top-level scope.
- Named imports/exports use braces and must match names unless aliased. A module can have one default export.

Web storage

```
localStorage.setItem('theme', 'dark');
const theme = localStorage.getItem('theme'); // string or null
localStorage.removeItem('theme');

localStorage.setItem('user', JSON.stringify(user));
const saved = JSON.parse(localStorage.getItem('user') ?? 'null');
```

Storage	Lifetime / scope
localStorage	persists for the origin until cleared

Storage	Lifetime / scope
sessionStorage	persists for the tab/session
cookies	small strings; can be sent with HTTP requests; security flags matter

Errors and debugging

```
try {
  riskyOperation();
} catch (error) {
  console.error(error.name, error.message);
} finally {
  releaseResource();
}

throw new TypeError('Expected a number');
```

- SyntaxError: invalid code grammar. ReferenceError: missing/invalid identifier access. TypeError: operation inappropriate for a value/type.
- Use breakpoints, step controls, watch expressions, the console, and Network panel. Do not rely only on scattered console.log calls.

11. Predict-the-Output Traps

1. `const a = [1]; const b = a; b.push(2); a.length`

Answer: 2. a and b reference the same array.

2. `typeof null`

Answer: 'object'. This is a historical language quirk.

3. `Boolean([])` and `Boolean({})`

Answer: Both true. Empty arrays and objects are truthy.

4. `[1, 2, 3].map(n => { n * 2 })`

Answer: [undefined, undefined, undefined]. Braces require an explicit return.

5. `[10, 2, 1].sort()`

Answer: [1, 10, 2]. Default sort is string-based.

6. `0 || 5` vs `0 ?? 5`

Answer: 5 vs 0. || reacts to all falsy values; ?? only to null/undefined.

7. `const x = user.address.city` when address is missing

Answer: TypeError. Use user.address?.city when absence is allowed.

8. Does `fetch('/missing')` reject solely because the server returns 404?

Answer: Usually no. Check response.ok/status.

9. `for (var i=0; i<3; i++) setTimeout(() => console.log(i))`

Answer: Logs 3 three times because var creates one function-scoped binding.

10. `parseInt('12px', 10)` vs `Number('12px')`

Answer: 12 vs NaN. parseInt stops at the first invalid digit; Number requires the whole string to be numeric.

12. Last-Minute Checklist

- Trace variable values and types line by line; do not mentally skip coercion.
- Check whether a method mutates its receiver and what it returns.
- Distinguish array index, element value, object key, and object value.
- Check block scope, function scope, closure capture, and TDZ before predicting output.
- For callbacks, check parameters, return value, and whether braces require explicit return.
- For DOM questions, check whether a selector can return null and whether the collection is live or static.
- For events, separate target from currentTarget and default action from propagation.
- For async questions, identify which expression is a Promise and where await/then handles it.
- For fetch, check HTTP status and remember JSON parsing is asynchronous.
- Prefer strict equality, const/let, textContent for untrusted text, and semantic browser APIs.

Scoring note: COITB's current public pages conflict on the JavaScript passing score: the badge page says 73% while the outline says 75%. Treat 75% as the safer preparation target and confirm the live requirement in your candidate portal.

Sources and Scope

Exam domains and weights: COITB, JavaScript Professional Developer Exam Outline, https://www.coitb.org/certification-resources/exam-outlines/javascript-professional-developer-outline (accessed 2026-09-30).

Exam expectations and badge details: COITB, JavaScript Professional Developer Certification, https://www.coitb.org/certification-resources/badges/javascript-professional-developer (accessed 2026-09-30).

Supporting technical reference: MDN Web Docs, JavaScript Guide, https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide (accessed 2026-09-30).

This is an independent study aid, not an official COITB publication or a reproduction of live exam questions. The published blueprint can change; verify current details before test day.